

Computer Networks



Overview of the Internet Protocol layering (Book 1 Ch 1)

Application Layer

- Application-Layer Paradigms
- Client-server applications
 - World Wide Web and HTTP
 - FTP
 - Electronic Mail
 - DNS
- Peer-to-peer paradigm
 - P2P Networks
- Case study: BitTorrent (Book 1 Ch 2)

① Introduction

- Providing Services
- Application-Layer Paradigms

② Standard Client-Server Applications

- World Wide Web and HTTP
- FTP
- Electronic Mail
- Domain Name System (DNS)

③ Peer-to-Peer Paradigm

- P2P Networks

File Transfer Protocol (FTP)

Definition

File Transfer Protocol (FTP) is the standard protocol provided by TCP/IP for copying a file from one host to another.

- File transfer may appear simple, but communicating systems can be different.
- Two systems may use different:
 - File name conventions
 - Data representations
 - Directory structures
- FTP provides a simple and elegant solution to these differences.

Why Use FTP?

Files can also be transferred using HTTP.

However, FTP is a better choice when:

- Transferring large files
- Transferring files in different formats
- Managing files and directories on a remote system

Main Purpose

Copy files from one host to another

Basic Model of FTP

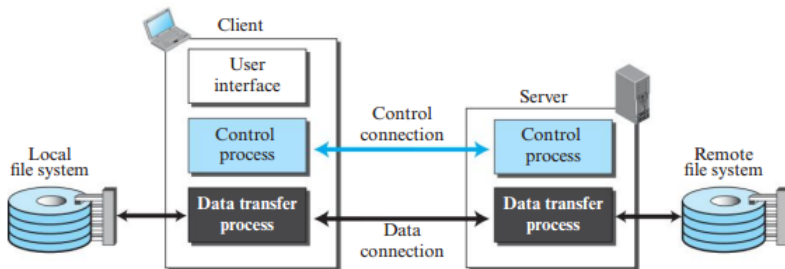
The FTP client contains three components:

- ① User interface
- ② Client control process
- ③ Client data transfer process

The FTP server contains two components:

- ① Server control process
- ② Server data transfer process

Figure 2.17 *FTP*



FTP Uses Two Connections

FTP uses two separate TCP connections.

Control Connection

- Created between the client and server control processes.
- Used for commands and responses.

Data Connection

- Created between the data transfer processes.
- Used for actual data transfer.

FTP = Control Connection + Data Connection

Why Does FTP Separate Commands and Data?

Separation of commands and data transfer makes FTP more efficient.

Control Connection

- Uses simple communication rules.
- Transfers one line of command or response at a time.

Data Connection

- Transfers different types of data.
- Requires more complex communication rules.

Key Idea

Commands and actual file data travel through separate connections.

Lifetimes of the Two FTP Connections

The two FTP connections have different lifetimes.

Control Connection

- Opens when the FTP session starts.
- Remains open during the entire interactive session.
- Closes when the user ends the FTP session.

Data Connection

- Opens for each file transfer activity.
- Closes after the transfer is completed.
- May open and close multiple times in one FTP session.

FTP Port Numbers

FTP uses two well-known TCP port numbers.

Port	Purpose
21	Control connection
20	Data connection

Port 21 → Commands and Responses

Port 20 → File Data

One FTP Session, Multiple Data Connections

During one FTP session:

- 1 The control connection is opened.
- 2 A data connection is opened for a file transfer.
- 3 The file is transferred.
- 4 The data connection is closed.
- 5 Another data connection may be opened for another transfer.
- 6 The control connection remains open throughout the session.

1 Control Connection + Multiple Data Connections

Control Connection

FTP uses the same general approach as TELNET for control communication.

- Uses the **NVT ASCII** character set.
- Communication occurs through:
 - Commands
 - Responses
- One command or response is sent at a time.
- Each line ends with:
 - Carriage return
 - Line feed



Commands and Responses

During the control connection:

Client $\xrightarrow{\text{Commands}}$ Server

Client $\xleftarrow{\text{Responses}}$ Server

- Commands are generated by the FTP client control process.
- Commands are written in uppercase ASCII.
- A command may or may not contain an argument.

General Command Format

COMMAND Argument

Common FTP Commands: Session and Navigation

Command	Argument	Description
USER	User ID	Sends user information
PASS	Password	Sends user password
QUIT	—	Logs out of the system
PWD	—	Displays current directory
CWD	Directory name	Changes directory
CDUP	—	Changes to parent directory
LIST	Directory name	Lists files or subdirectories

Common FTP Commands: File Operations

Command	Argument	Description
RETR	File name	Retrieves a file from server to client
STOR	File name	Stores a file from client to server
DELE	File name	Deletes a file
RNFR	Old file name	Identifies file to be renamed
RNTO	New file name	Renames the file
ABOR	—	Aborts the previous command

Common FTP Commands: Directory and Connection

Command	Argument	Description
MKD	Directory name	Creates a new directory
RMD	Directory name	Deletes a directory
PORT	Port identifier	Client chooses a port
PASV	—	Server chooses a port

FTP Commands for Data Representation

Command	Argument	Purpose
STRU	F, R, or P	Defines data organization
TYPE	A, E, or I	Defines file type
MODE	S, B, or C	Defines transmission mode

- **STRU**: File, Record, or Page
- **TYPE**: ASCII, EBCDIC, or Image
- **MODE**: Stream, Block, or Compressed

Every FTP command generates at least one response.
A response contains two parts:

Three-Digit Code + Text

- The numeric part defines the response code.
- The text provides:
 - Additional explanation
 - Required parameters

Understanding the FTP Response Code

An FTP response code contains three digits.

First Digit

Defines the status of the command.

Second Digit

Defines the area in which the status applies.

Third Digit

Provides additional information.

XYZ

X = Status, Y = Area, Z = Additional Information

Common FTP Responses

Code	Meaning	Code	Meaning
125	Data connection open	250	File action OK
150	File status OK	331	User name OK; password needed
200	Command OK	425	Cannot open data connection
220	Service ready	450	File action not taken
221	Service closing	452	Insufficient storage
225	Data connection open	500	Unrecognized command
226	Closing data connection	501	Error in parameters
230	User login OK	530	User not logged in

- The data connection is used for actual file transfer.
- The server uses well-known port:

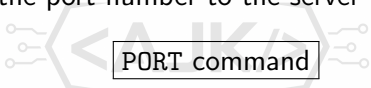


- The creation of the data connection is different from the creation of the control connection.
- The client initiates the process of creating the data connection.

Creating the Data Connection

The data connection is created in three steps:

- 1 The client issues a **passive open** using an ephemeral port.
- 2 The client sends the port number to the server using the:



PORT command

- 3 The server issues an **active open** using:
 - Server port 20
 - Client's ephemeral port number

Why Does the Client Choose the Data Port?

- The client issues the commands that request file transfers.
- Therefore, the client prepares a temporary port for the data connection.
- The client informs the server of this port using:



- The server then establishes the data connection from port 20.

Server Port 20 → Client Ephemeral Port

Preparing Data for Transfer

The purpose of the data connection is to transfer files.

Before sending a file, the client must define:

- 1 File type
- 2 Data structure
- 3 Transmission mode

Why?

FTP must solve differences between heterogeneous systems.

File Type + Data Structure + Transmission Mode

FTP supports three interpretations of data structure:

- ① **File Structure**
- ② **Record Structure**
- ③ **Page Structure**



Purpose

The data structure defines how the contents of a file are organized during transfer.

File Structure

The file is treated as a continuous stream of bytes with no internal structure.

- This is the **default structure**.
- FTP does not interpret internal records or pages.
- The data are transferred as:

Continuous Stream of Bytes

Record and Page Structures

Record Structure

- The file is divided into records.
- Can be used only with text files.

Page Structure

- The file is divided into pages.
- Each page has:
 - A page number
 - A page header
- Pages can be accessed randomly or sequentially.

FTP File Types

FTP can transfer three file types:

- 1 **ASCII File**
- 2 **EBCDIC File**
- 3 **Image File**

TYPE	Argument	File Type
	A	ASCII
	E	EBCDIC
	I	Image

Command

```
TYPE A, TYPE E, TYPE I
```

FTP Transmission Modes

FTP supports three transmission modes:

- 1 **Stream Mode**
- 2 **Block Mode**
- 3 **Compressed Mode**

MODE	Argument	Mode
	S	Stream
	B	Block
	C	Compressed

Stream Mode

Stream Mode

Data are delivered from FTP to TCP as a continuous stream of bytes.

- This is the **default transmission mode**.
- Data flow continuously:

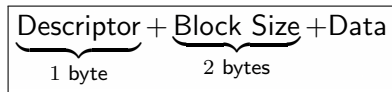
FTP $\longrightarrow$ TCP

- No block header is added by the FTP transmission mode.

Block Mode

In block mode, data are delivered from FTP to TCP in blocks. Each block is preceded by a **3-byte header**.

Header Field	Size
Block Descriptor	1 byte
Block Size	2 bytes



Compressed Mode

Compressed Mode

Data are compressed before being transferred over the data connection.

- Identified by:
MODE C
- Can reduce the amount of data transmitted.
- Useful when compression is beneficial for the transferred data.

File Transfer in FTP

File transfer occurs over the **data connection**.

However, it is controlled by commands sent over the **control connection**.

Control Connection → Controls the Transfer

Data Connection → Carries the Data

Three Types of FTP File Transfer

In FTP, file transfer can mean one of three operations:

① **Retrieving a File**

Server → Client

② **Storing a File**



Client → Server

③ **Directory Listing**

Server → Client

Commands Used for File Transfer

Command	Operation	Direction
RETR	Retrieve file	Server → Client
STOR	Store file	Client → Server
LIST	Directory listing	Server → Client

Remember

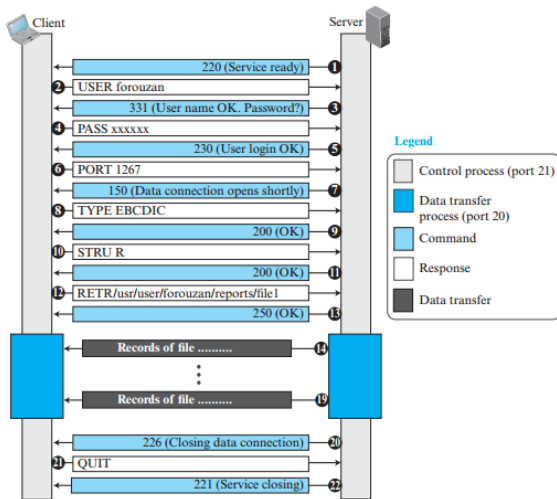
Commands travel through the control connection; transferred content travels through the data connection.

Example 2.11: Retrieving a File

The client retrieves one file from the server.

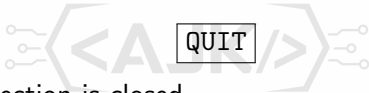
- The control connection remains open throughout the session.
- The data connection is opened for the transfer.
- The file is assumed to be transferred in six sections.
- After all data are transferred:
 - The server announces completion.
 - The data connection is closed.

Figure 2.18 Example 2.11



Example 2.11: Ending the FTP Session

After the file has been transferred:

- 1 The server control process announces that the transfer is complete.
- 2 The client has no more files to retrieve.
- 3 The client sends:

- 4 The control connection is closed.

Key Point

The data connection ends after the transfer, but the control connection ends only when the FTP session is terminated.

Example 2.12: Starting an FTP Session

```
$ ftp voyager.deanza.fhda.edu
```

```
Connected to voyager.deanza.fhda.edu.
```

```
220 (vsFTPd 1.2.1)
```

```
530 Please login with USER and PASS.
```

```
Name: forouzan
```

```
331 Please specify the password.
```

```
Password: *****
```

```
230 Login successful.
```

Observation

The server uses response codes to indicate the status of each step.

Example 2.12: Directory Listing

Remote system type is UNIX.

Using binary mode to transfer files.

227 Entering Passive Mode

150 Here comes the directory listing.

drwxr-xr-x business

drwxr-xr-x personal

drwxr-xr-x school

226 Directory send OK.

- The directory listing is transferred over the data connection.
- The control connection remains active.

Example 2.12: Ending the Session

```
ftp> quit  
221 Goodbye.
```

- The client sends the QUIT command.
- The server responds with:

221 Goodbye

- The FTP control connection is terminated.

Security Problems in FTP

FTP was designed when network security was not a major concern.

Major Security Problem

FTP sends sensitive information in plaintext.

- User passwords are sent unencrypted.
- File data are also transferred in plaintext.
- An attacker may intercept:
 - Login credentials
 - Transferred files

Securing FTP

A security layer can be added between FTP and TCP.

FTP → Secure Socket Layer → TCP

This secure form is called:



- Protects communication from plaintext interception.
- Other secure file-transfer mechanisms can also use SSH.

FTP: Key Takeaways

- FTP copies files between hosts.
- FTP separates:
 - Commands and responses
 - Actual data transfer
- Port 21 is used for control.
- Port 20 is used for data.
- The control connection remains open for the session.
- Data connections open and close for transfer activities.
- FTP supports different:
 - Data structures
 - File types
 - Transmission modes
- Traditional FTP is insecure because it uses plaintext.